

Informe de Práctica: Proyecto ASP.NET Core MVC

Asignatura: Aplicaciones Web

Estudiante: [Zaida Melissa Taipe Mora]

1. Objetivo de la Práctica

Desarrollar una aplicación web funcional empleando el marco de trabajo ASP.NET Core MVC. La práctica se centra en la configuración inicial del proyecto, la estructuración del controlador de productos, y la aplicación de mecanismos robustos para la validación y seguridad de los datos. Entre los principales hitos se incluyen la creación de un modelo con Data Annotations, la construcción de vistas Razor que incorporen AntiForgeryToken, y la respectiva verificación de la validación a nivel de servidor.

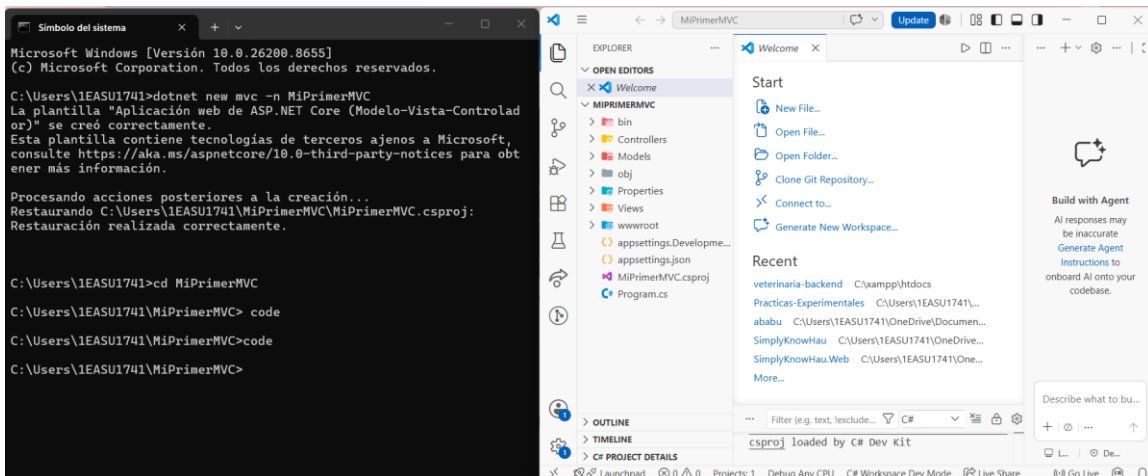
2. Herramientas y Requisitos

- .NET SDK 10.0
- Entorno de desarrollo (Visual Studio 2022 o Visual Studio Code).

3. Desarrollo y Paso a Paso

3.1. Creación del Proyecto MVC

El desarrollo se inició estableciendo la estructura base del proyecto mediante la interfaz de línea de comandos (CLI) de .NET. Se ejecutaron las instrucciones correspondientes para inicializar la plantilla MVC y acceder al directorio de trabajo. (dotnet new mvc -n MiPrimerMVC cd MiPrimerMVC)



(Figura 1: Creación del proyecto desde la terminal)

3.2. Modelo de Datos y Data Annotations

Posteriormente, se diseñó la entidad principal del negocio, la clase Producto, ubicada dentro del directorio Models. Para garantizar la integridad y calidad de la información ingresada por los usuarios, se incorporaron Data Annotations tales como [Required] o [Range]. Estas directivas permiten establecer las reglas de validación que debe cumplir cada propiedad.

```
using System.ComponentModel.DataAnnotations;
namespace MiPrimerMVC.Models
{
    public class Producto
    {
        public int Id { get; set; }

        [Required(ErrorMessage = "El nombre es obligatorio.")]
        [StringLength(100, MinimumLength = 3, ErrorMessage = "El nombre debe tener entre 3 y 100 caracteres.")]
        [Display(Name = "Nombre del producto")]
        public string Nombre { get; set; } = string.Empty;

        [Required(ErrorMessage = "La descripción es obligatoria.")]
        [StringLength(500, ErrorMessage = "La descripción no puede exceder 500 caracteres.")]
        [Display(Name = "Descripción")]
        public string Descripcion { get; set; } = string.Empty;

        [Required(ErrorMessage = "El precio es obligatorio.")]
        [Range(0.01, 100000, ErrorMessage = "El precio debe estar entre 0.01 y 100000.")]
        [DataType(DataType.Currency)]
        [Display(Name = "Precio (USD)")]
        public decimal Precio { get; set; }

        [Required(ErrorMessage = "El stock es obligatorio.")]
        [Range(0, int.MaxValue, ErrorMessage = "El stock no puede ser negativo.")]
        [Display(Name = "Stock disponible")]
        public int Stock { get; set; }

        [EmailAddress(ErrorMessage = "El correo del proveedor no tiene un formato válido.")]
        [Display(Name = "Correo del proveedor")]
        public string? CorreoProveedor { get; set; }
    }
}
```

3.3. Configuración del ProductoController

Para gestionar las solicitudes y respuestas de la aplicación, se implementó el ProductoController en el directorio Controllers. En este archivo se programaron dos acciones fundamentales:

- Index: Destinada a presentar un catálogo o lista de los productos registrados.
- Create: Conformada por dos métodos; uno para responder a peticiones GET (desplegando el formulario de ingreso) y otro para gestionar las peticiones POST (procesando los datos enviados).

```
using Microsoft.AspNetCore.Mvc;
using MiPrimerMVC.Models;

namespace MiPrimerMVC.Controllers
{
    public class ProductoController : Controller
    {
        private static readonly List<Producto> _productos = new();

        [HttpGet]
        public IActionResult Index()
        {
            return View(_productos);
        }

        [HttpGet]
        public IActionResult Create()
        {
            return View();
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        public IActionResult Create(Producto producto)
        {
            if (!ModelState.IsValid)
            {
                return View(producto);
            }

            producto.Id = _productos.Count + 1;
            _productos.Add(producto);

            return RedirectToAction(nameof(Index));
        }
    }
}
```

3.4. Diseño de Vistas Razor y AntiForgeryToken

A continuación, se construyeron las vistas correspondientes dentro de Views/Producto. En la vista Create.cshtml, se configuró el formulario html haciendo uso de los Tag Helpers de

ASP.NET Core. Es relevante destacar la inclusión y funcionamiento del AntiForgeryToken en el formulario, un mecanismo esencial que protege a la aplicación contra vulnerabilidades de falsificación de solicitudes entre sitios (CSRF).

```
@model MiPrimerMVC.Models.Producto
@{
    ViewData["Title"] = "Crear Producto";
}

<h1>Crear Producto</h1>
<hr />
<div class="row">
    <div class="col-md-6">
        @* asp-antiforgery="false" desactiva el token automático del tag helper
           para poder mostrarlo explícitamente con @Html.AntiForgeryToken(). *@
        <form asp-action="Create" method="post" asp-antiforgery="false">
            @Html.AntiForgeryToken()

            <div asp-validation-summary="All" class="text-danger mb-3"></div>

            <div class="mb-3">
                <label asp-for="Nombre" class="form-label"></label>
                <input asp-for="Nombre" class="form-control" />
                <span asp-validation-for="Nombre" class="text-danger"></span>
            </div>

            <div class="mb-3">
                <label asp-for="Descripcion" class="form-label"></label>
                <textarea asp-for="Descripcion" class="form-control" rows="3"></textarea>
                <span asp-validation-for="Descripcion" class="text-danger"></span>
            </div>

            <div class="mb-3">
                <label asp-for="Precio" class="form-label"></label>
                <input asp-for="Precio" class="form-control" />
                <span asp-validation-for="Precio" class="text-danger"></span>
            </div>

            <div class="mb-3">
                <label asp-for="Stock" class="form-label"></label>
                <input asp-for="Stock" class="form-control" />
                <span asp-validation-for="Stock" class="text-danger"></span>
            </div>

            <div class="mb-3">
```

```

        <label asp-for="CorreoProveedor" class="form-label"></label>
        <input asp-for="CorreoProveedor" class="form-control" />
        <span asp-validation-for="CorreoProveedor" class="text-danger"></span>
    </div>

    <button type="submit" class="btn btn-primary">Guardar</button>
    <a asp-action="Index" class="btn btn-secondary">Volver al listado</a>
</form>
</div>
</div>

```

@* No se incluye _ValidationScriptsPartial: así la validación ocurre en el servidor y se puede comprobar el round-trip. *@

The screenshot shows a web browser window with the URL 'localhost:5164/Producto/Create'. The page has a header with 'MiPrimerMVC' and navigation links 'Productos' and 'Privacy'. The main content area is titled 'Crear Producto'. It contains five form fields: 'Nombre del producto' (text), 'Descripción' (text area), 'Precio (USD)' (text), 'Stock disponible' (text), and 'Correo del proveedor' (text). Below the fields are two buttons: 'Guardar' (orange) and 'Volver al listado' (gray). The footer shows '© 2026 - MiPrimerMVC - Privacy'.

(Figura 2: Código de la vista Create.cshtml con el formulario)

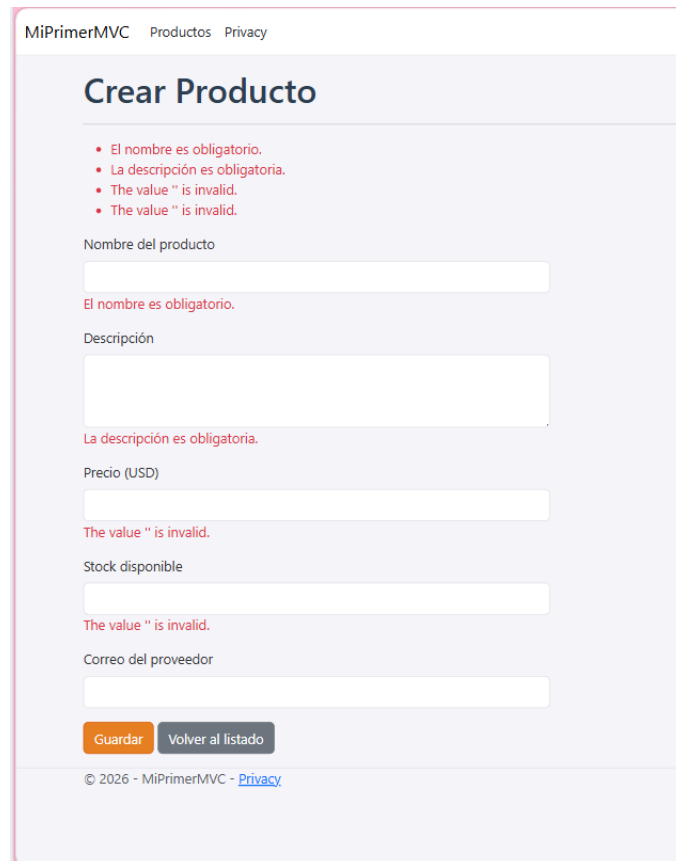
3.5. Validación Server-Side (ModelState)

La verificación de los datos no se limitó al navegador; la validación a nivel de servidor es el pilar de la seguridad de este módulo. Dentro del método Create de tipo POST, se examinó la propiedad ModelState.IsValid. Este flujo condicional verifica si la información suministrada aprueba las reglas de las Data Annotations antes de cualquier registro.

4. Pruebas y Resultados de Validación

Una vez estructurado el código, se ejecutó la aplicación para validar el flujo completo. Se simularon distintos escenarios para observar el comportamiento de las validaciones en tiempo de ejecución.

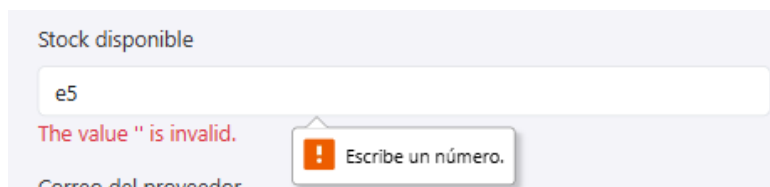
- Caso 1: Envío de formulario vacío. Se dispararon las alertas correspondientes a los campos obligatorios.



The screenshot shows a web application titled 'MiPrimerMVC' with a navigation bar containing 'Productos' and 'Privacy'. The main heading is 'Crear Producto'. Below the heading, there are four red bullet points indicating validation errors: 'El nombre es obligatorio.', 'La descripción es obligatoria.', 'The value "" is invalid.', and 'The value "" is invalid.'. The form contains five input fields: 'Nombre del producto', 'Descripción', 'Precio (USD)', 'Stock disponible', and 'Correo del proveedor'. Each field has a corresponding red error message below it: 'El nombre es obligatorio.', 'La descripción es obligatoria.', 'The value "" is invalid.', 'The value "" is invalid.', and 'The value "" is invalid.'. At the bottom of the form, there are two buttons: 'Guardar' (orange) and 'Volver al listado' (gray). The footer shows '© 2026 - MiPrimerMVC - [Privacy](#)'.

(Figura 3: Resultados de validación por campos en blanco)

- Caso 2: Envío de datos con formato incorrecto o fuera de rango (ej. precios inválidos).



The screenshot shows a close-up of the 'Stock disponible' input field. The field contains the text 'e5'. Below the field, there is a red error message: 'The value "" is invalid.'. A tooltip with an orange exclamation mark icon and the text 'Escribe un número.' is displayed over the error message. Below the field, the text 'Correo del proveedor' is partially visible.

(Figura 4: Mensajes de error por datos inválidos)

- Caso 3: Registro exitoso. Una vez que la información cumplió con todas las reglas, la aplicación guardó el producto y redireccionó a la lista.



(Figura 5: Visualización del listado de productos registrados)

5. Repositorio del Proyecto

El código fuente completo y los recursos del proyecto se encuentran versionados y disponibles para revisión en el siguiente enlace: <https://github.com/Zaida-tm18/MiPrimerMVC.git>

6. Conclusiones

- La estructura MVC en ASP.NET Core permite mantener un orden lógico entre los datos (Modelos), la lógica de negocio (Controladores) y la interfaz de usuario (Vistas).
- La integración de Data Annotations resulta altamente eficiente para centralizar las reglas de validación en la definición de la clase, reduciendo redundancias y código disperso.
- A través de la instrucción ModelState.IsValid, se demostró que el servidor funciona como la barrera principal de validación, garantizando la seguridad sin depender únicamente del entorno del cliente.
- La habilitación del AntiForgeryToken en los formularios representa una práctica esencial y obligatoria para prevenir inyecciones y falsificaciones de identidad web.

7. Anexos

```

Simbolo del sistema - dotnet x + v
Microsoft Windows [Versión 10.0.26200.8655]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\1EASU1741>dotnet restore
MSBUILD : error MSB1003: Especifique un archivo de proyecto o de solución. El directorio de trabajo actual no contiene u
n archivo de proyecto ni de solución.

C:\Users\1EASU1741>cd MiPrimerMVC

C:\Users\1EASU1741\MiPrimerMVC>dotnet restore
Restauración completada (0,3s)

Compilación realizado correctamente en 0,6s

C:\Users\1EASU1741\MiPrimerMVC>dotnet run
Usando la configuración de inicio de C:\Users\1EASU1741\MiPrimerMVC\Properties\launchSettings.json...
Compilando...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5164
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\Users\1EASU1741\MiPrimerMVC
warn: Microsoft.AspNetCore.HttpsPolicy.HttpsRedirectionMiddleware[3]
      Failed to determine the https port for redirect.
  
```

